Experiment No: 07

Experiment Title: Study & Implementation of Subqueries in SQL with Suitable Examples

Topics to be covered:

- Subqueries (Single-row, Multi-row, Correlated)

Subqueries in SQL

Subqueries are queries embedded within another SQL query. They are typically used to perform intermediate steps for filtering, comparison, or data retrieval.

Subqueries are queries nested inside another query. They can be used in the SELECT, FROM, or WHERE clause.

Types of Subqueries:

- **Single-row Subquery:** Returns a single row.
- **Multi-row Subquery:** Returns multiple rows.
- **Correlated Subquery:** References columns from the outer query.

1.Single-Row Subquery

Definition:

Returns **one row** and is used with comparison operators like =, >, <, etc.

Syntax:

```
SELECT column1, column2
FROM table
WHERE column3 operator (SELECT aggregate_function(column) FROM
table);
```

Example:

```
SELECT first_name, salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees);
```

Sample employees Table:

id	first_name	salary	department
1	Alice	60000	Engineering
2	Bob	75000	Engineering
3	Carol	80000	Marketing
4	David	50000	Sales
5	Eva	70000	Marketing

- AVG(salary) = (60000 + 75000 + 80000 + 50000 + 70000) / 5 = **67000**

Output:

first_name	salary
Bob	75000
Carol	80000
Eva	70000

2.Multi-Row Subquery

Definition:

Returns **multiple rows** and is used with operators like IN, ANY, ALL.

Syntax:

```
SELECT column1, column2
FROM table
WHERE column3 IN (SELECT column FROM table WHERE condition);
```

Example:

```
SELECT first_name, department
FROM employees
WHERE department IN (
  SELECT department FROM employees WHERE salary > 70000);
```

Subquery Result:

- Subquery: SELECT department FROM employees WHERE salary > 70000
- Employees with salary > 70000: Bob (75000, Engineering), Carol (80000, Marketing)
- Departments: **Engineering**, **Marketing**

Output:

first_name	department
Alice	Engineering
Bob	Engineering
Carol	Marketing
Eva	Marketing

David (Sales) is excluded as his department isn't in the subquery result.

3.Correlated Subquery

Definition:

A subquery that **references a column from the outer query**. It's evaluated **once per row** of the outer query.

Syntax:

```
SELECT outer.column1, outer.column2
FROM table outer
WHERE outer.column3 > (
```

```
  SELECT AVG(inner.column)
  FROM table inner
  WHERE outer.related_column = inner.related_column
);
```

Example:

```
SELECT e1.first_name, e1.salary
FROM employees e1
WHERE salary > (
  SELECT AVG(salary)
  FROM employees e2
  WHERE e1.department = e2.department
);
```

Average Salaries per Department:

- **Engineering**: (60000 + 75000) / 2 = 67500
- **Marketing**: (80000 + 70000) / 2 = 75000
- **Sales**: 50000

Row Evaluation:

e1.first_name	e1.salary	e1.department	DeptAvg Salary	Condition Met?
Alice	60000	Engineering	67500	❌ No
Bob	75000	Engineering	67500	✅ Yes
Carol	80000	Marketing	75000	✅ Yes
David	50000	Sales	50000	❌ No
Eva	70000	Marketing	75000	❌ No

Output:

first_name	salary
Bob	75000
Carol	80000

Summary Table:

Type	Returns	Used With Operators	Subquery Example
Single-row	1 row	=, >, <	salary > (SELECT AVG(salary))
Multi-row	Multiple rows	IN, ANY, ALL	department IN (SELECT department WHERE salary > X)
Correlated	Per-row eval	Any (complex logic)	salary > (SELECT AVG(salary) WHERE dept = dept)

Lab Tasks-1:

Sample table:**Employees**

employee_id	first_name	last_name	salary	hire_date	department	date_of_birth	gender	city
1	John	Doe	60000	2020-01-15	IT	1985-08-12	M	New York
2	Jane	Smith	70000	2019-03-23	HR	1990-05-22	F	Chicago
3	Tom	Brown	75000	2021-07-01	Sales	1988-12-04	M	Boston
4	Alice	Green	85000	2018-11-05	IT	1987-07-19	F	Miami
5	Bob	White	50000	2022-09-12	Marketing	1995-03-30	M	Dallas

Questions:

1. Find employees who earn more than the average salary.
2. List employees whose salary is higher than all employees in Marketing.
3. Display departments that have at least one employee earning above 70,000.
4. Find the employee(s) with the highest salary.
5. Find employees who were hired before the average hire date.
6. List employees from the same city as employees in HR.
7. Find employees whose salary is greater than the average salary of their department (correlated).
8. Find employees whose salary is equal to the second highest salary.
9. List employees older than the oldest HR employee.
10. Find employees working in departments where the average salary is greater than 65,000.

Lab Task-2

Sample Table: **Customers**

CustomerID	Name	City	Age	TotalPurchase
1	John Smith	New York	28	1200.50
2	Sarah Johnson	Los Angeles	34	850.75
3	Michael Lee	Chicago	22	640.00
4	Emma Wilson	Houston	29	980.10
5	Daniel Kim	Phoenix	41	1500.00
6	Olivia Brown	Dallas	36	1330.20
7	James Davis	New York	27	700.00
8	Emily Garcia	Chicago	31	920.75
9	David Anderson	Los Angeles	39	1010.00
10	Sophia Martinez	Houston	24	610.00

Questions:

1. Find customers who spent more than the average purchase amount.
2. Find the youngest customer using a subquery.
3. List customers from the same city as Emma Wilson.
4. Find customers who spent more than Michael Lee.
5. Find the second highest purchase amount.
6. List customers older than the average age.
7. Find customers living in the same city as any customer under 25.
8. Find customers who spent less than the minimum purchase in Los Angeles.
9. List customers who have the highest purchase in their city (correlated).
10. Find customers whose purchase is above the overall median purchase.

Lab Task-3:

Tables: **Employees** + **Customers**

1. Find employees who live in the same city as at least one customer.
2. List customers whose purchase is higher than any employee's salary ÷ 100.
3. Find all cities that have both customers and employees.
4. List employees older than the oldest customer.
5. Find customers from cities where employees earn above the company average salary.
6. List employees who share the same first name with any customer.
7. Find customers who live in a city where at least one IT employee works.
8. Find customers whose purchases are greater than the average employee salary ÷ 100.
9. List employees who earn more than 100 times the lowest customer purchase.
10. Find employees and customers who both live in New York.

Lab Task-4:
Sample Tables:

1. Customers

CustomerID	Name	City	Age	TotalPurchase
1	John Smith	New York	28	1200.50
2	Sarah Johnson	Los Angeles	34	850.75
3	Michael Lee	Chicago	22	640.00
4	Emma Wilson	Houston	29	980.10

2. TopCustomers (Empty initially)

CustomerID	Name	TotalPurchase
(empty)	*(empty)*	*(empty)*

- Subqueries with INSERT

Insert customers with purchases above average into TopCustomers.

```
INSERT INTO TopCustomers (CustomerID, Name, TotalPurchase)
SELECT CustomerID, Name, TotalPurchase
FROM Customers
WHERE TotalPurchase> (
    SELECT AVG(TotalPurchase) FROM Customers
);
```

- Subqueries with UPDATE

Increase the TotalPurchase by 10% for customers younger than the average age.

```
UPDATE Customers
SET TotalPurchase = TotalPurchase * 1.10
WHERE Age < (
    SELECT AVG(Age) FROM Customers
);
```

This targets only customers below the average age using a subquery.

- Subqueries with DELETE

Delete customers who spent **less than the minimum purchase made by anyone in Los Angeles**.

```
DELETE FROM Customers
WHERE TotalPurchase< (
    SELECT MIN(TotalPurchase)
    FROM Customers
    WHERE City = 'Los Angeles'
);
```